

Enchanted Library

Dokumentacja projektu

Autorki	Małgorzata Pytlak, Julia Radosz
Przedmiot	Programowanie w języku Python — Projekt II
Framework	Flask (Python 3.10+)
Baza danych	SQLite (instance/basniowa_biblioteka.db)
API	Open Library (openlibrary.org)
Rok akademicki	2025/26

1. Opis aplikacji

Enchanted Library to webowy mini-Goodreads dla miłośniczek książek. Katalog jest pobierany na żywo z darmowego Open Library API — w bazie lokalnej zapisujemy tylko konta użytkowników, recenzje oraz osobiste półki (To Read, Reading, Read). Interfejs jest w stylu fairycore (różowo-kremowa paleta, czcionka Cormorant Garamond, szklane karty).

2. Uruchomienie

2.1 Wymagania

- Python 3.10 lub nowszy,
- pip,
- zależności z requirements.txt: flask, flask-sqlalchemy, flask-login, Werkzeug, requests, pytest.

2.2 Kroki

Instalacja zależności:

```
pip install -r requirements.txt
```

Jednorazowa inicjalizacja bazy (tworzy konta testowe z zahashowanymi hasłami):

```
python -c "from app import create_app, db; from app.models import User;
from Werkzeug.security import generate_password_hash;
app = create_app(); ctx = app.app_context(); ctx.push();
db.drop_all(); db.create_all();
db.session.add(User(username='admin', email='admin@lib.pl',
    password_hash=generate_password_hash('puchatek123'), is_admin=True));
db.session.add(User(username='kopciuszek', email='k@lib.pl',
    password_hash=generate_password_hash('pantofelek'), is_admin=False));
db.session.commit(); print('Gotowe!')"
```

Uruchomienie serwera:

```
python run.py
```

Aplikacja nasłuchuje na <http://127.0.0.1:5000>.

2.3 Konta testowe

Rola	Login	Hasło
Admin	admin	puchatek123
Użytkownik	kopciuszek	pantofelek

3. Architektura

Aplikacja jest klasycznym projektem Flaskowym z wzorcem fabryki aplikacji (create_app) i podziałem funkcjonalności na blueprinty.

run.py	– punkt wejścia, tworzy app i odpala serwer
config.py	– konfiguracja (klucz sesji, ścieżka bazy)
app/__init__.py	– fabryka create_app, rejestracja blueprintów
app/models.py	– modele SQLAlchemy (User, Book, Review, UserBookStatus)
app/errors.py	– handlers 404 i 500
app/utils.py	– klient Open Library (search + subject)
app/auth/routes.py	– blueprint 'auth' (login, register, logout, admin)
app/books/routes.py	– blueprint 'books' (katalog, półka, recenzje)
app/templates/	– szablony Jinja2
app/static/	– CSS + grafiki gatunków
tests/	– testy pytest z bazą in-memory

3.1 Blueprinty

- auth (prefix /auth) — logowanie, rejestracja, wylogowanie, panel administratora, usuwanie recenzji i użytkowników.
- books (prefix /) — strona główna z katalogiem, szczegóły książki, operacje na półce i recenzjach, profil użytkownika.

3.2 Przepływ danych

1. Użytkownik wchodzi na / lub wyszukuje tytuł.
2. Widok wywołuje fetch_books_by_query lub fetch_books_by_subject z app/utils.py, która odpytuje openlibrary.org i zwraca listę dict-ów.
3. Szablon renderuje wyniki. Nie są one cache'owane ani zapisywane w bazie.
4. Dopiero gdy użytkownik doda książkę do półki (/update_status) lub napisze recenzję (/add_review), rekord Book trafia do lokalnej bazy.
5. Strona szczegółów (/book/<ol_id>) pobiera works, editions i dane autora z Open Library bezpośrednio w widoku — kilkanaście żądań HTTP na odsłone, brak cache.

3.3 Autoryzacja

- Sesyjna, oparta o Flask-Login. Po zalogowaniu serwer ustawia ciasteczko sesji w przeglądarce.
- Trasy wymagające zalogowania są oznaczone dekoratorem @login_required. Nieautoryzowany dostęp → przekierowanie na /auth/login.
- Kontrola uprawnień admina odbywa się inline wewnątrz widoków: if not current_user.is_admin: flash(...); return redirect(...).

3.4 Hasła

- Rejestracja używa `werkzeug.security.generate_password_hash`.
- Logowanie weryfikuje przez `check_password_hash`.
- W bazie trzymany jest tylko hash — nawet admin nie widzi hasła użytkownika.

4. Model danych

Tabela	Kolumny	Uwagi
user	id, username (UQ), email (UQ), password_hash, is_admin	UserMixin z Flask-Login
book	id, google_id (UQ), title, author, genre, year, description, cover_url, publisher, page_count	google_id to OLID z Open Library
review	id, rating, content, timestamp, user_id, book_id	backref: author, book
user_book_status	id, user_id, book_id, status	status = 'To Read' / 'Reading' / 'Read'

5. Uprawnienia

Rola	Katalog / szczegóły	Półka i recenzje	Panel admina
Gość (niezalogowany)	Tak	Nie	Nie
Zalogowany użytkownik	Tak	Tak	Nie
Administrator	Tak	Tak	Tak

Administrator może usuwać dowolne recenzje oraz dowolnych użytkowników (wraz z ich recenzjami i wpisami półki).

6. Testy

Uruchomienie:

```
pytest tests/ -v
```

Testy używają bazy SQLite w pamięci, a wywołania do Open Library są w całości mockowane (`unittest.mock.patch` oraz `monkeypatch`).

Plik	Zakres
conftest.py	TestingConfig, fixture app, client, db, user, admin, zalogowani klienci
test_auth.py	login, rejestracja, wylogowanie, panel admina, operacje delete
test_books.py	katalog, szczegóły, /update_status, /profile, /add_review
test_models.py	User (unikalność, is_admin), Book, Review, UserBookStatus
test_utils.py	Open Library — szczególne ścieżki, filtry, brak sieci
test_errors.py	404, 500, rollback db.session w handlerze 500

7. Podział pracy

Pracę nad projektem podzieliliśmy na dwa naturalne filary. AI (Claude) pomagało nam w zadaniach pomocniczych — głównie stylizacji, obsłudze błędów oraz redakcji tekstu dokumentacji — aby nie odbierać nam autorstwa części merytorycznej.

7.1 Małgorzata Pytlak (Gosia)

Odpowiedzialna za obszar użytkownika i administracji:

- `app/auth/routes.py` — pełny blueprint `auth`: logowanie, rejestracja (walidacja długości hasła, zgodności haseł, unikalności `username` i `e-mail`), wylogowanie, panel administratora, operacje `delete_review` i `delete_user` (kaskadowe usuwanie danych użytkownika).
- `app/auth/__init__.py` — konstrukcja blueprintu `auth`.
- `app/models.py` — modele `User` (wraz z `UserMixin` i `load_user`) oraz `Review`.
- Szablony: `login.html`, `register.html`, `admin.html`, `profile.html`.
- Logika seedu kont testowych w `run.py` (utworzenie admina i zwykłego użytkownika przy pierwszym uruchomieniu).
- Testy: `tests/test_auth.py` oraz części `tests/test_models.py` dotyczące `User` i `Review`.

7.2 Julia Radosz

Odpowiedzialna za katalog, integrację z Open Library i szkielet aplikacji:

- `app/__init__.py` — fabryka `create_app`, inicjalizacja `SQLAlchemy` i `Flask-Login`, rejestracja blueprintów.
- `app/books/routes.py` — pełny blueprint `books`: strona główna z bestsellerami i kategoriami, wyszukiwanie, widok szczegółów książki (pobieranie `works/editions/autora` z Open Library), `/update_status`, `/profile`, `/add_review`.
- `app/books/__init__.py` — konstrukcja blueprintu `books`.
- `app/utils.py` — klient Open Library: `fetch_books_by_query` (Search API) oraz `fetch_books_by_subject` (Subject API), wraz z filtrami (max 2 książki na autora, czarna lista nudnych tytułów, wymóg okładki).
- `app/models.py` — modele `Book` i `UserBookStatus` wraz z relacjami.
- Szablony: `base.html` (layout, nawigacja, flash), `index.html` (strona główna + karty gatunków), `book_details.html` (strona szczegółów z formularzami półki i recenzji).
- `run.py` — punkt wejściowy i inicjalizacja bazy.
- Testy: `tests/conftest.py` (fixture'y), `tests/test_books.py`, `tests/test_utils.py` oraz części `tests/test_models.py` dotyczące `Book` i `UserBookStatus`.

7.3 Wsparcie AI (Claude)

AI zostało użyte do zadań pomocniczych, niemerytorycznych — takich, które nie wpływają na logikę aplikacji. W kodzie źródłowym wkład AI jest celowo ograniczony do warstwy prezentacyjnej i obsługi błędów:

- `app/static/css/fairy_tale.css` — stylizacja `fairycore` (paleta kolorów, typografia, szklane karty, responsywność).
- `app/static/images/` — dobór i przygotowanie ilustracji dla kart gatunków (Fantasy, Thriller, Romance, Horror, History, Science).
- `app/errors.py` — rejestracja handlerów 404 i 500 (krótki plik).
- `app/templates/errors/404.html`, `500.html` — szablony stron błędów.
- `config.py` — minimalna klasa `Config` (`SECRET_KEY`, `URI` bazy, `SQLALCHEMY_TRACK_MODIFICATIONS`).
- `requirements.txt` — lista pakietów.
- `tests/test_errors.py` — testy handlerów błędów.

Dodatkowo AI pomogło przy materiałach towarzyszących (poza kodem aplikacji):

- README.txt — redakcja i formatowanie opisu uruchomienia, listy funkcji i kont testowych (treść merytoryczna pochodzi z przewodnika napisanego przez autorki).
- dokumentacja.pdf (niniejszy dokument) — redakcja tekstu sekcji opisowych, tabele, formatowanie PDF oraz skrypt generujący. Informacje merytoryczne (podział pracy, architektura, lista testów) zostały podane przez autorki.
- tests/conftest.py — scaffolding fixture'ów (TestingConfig, app, client, logged_in_client); asercje w poszczególnych plikach testowych pisane przez autorki.

Cała logika biznesowa (auth, blueprinty, modele, integracja z API, recenzje, uprawnienia admina oraz większość testów) została napisana przez autorki.

8. Znane ograniczenia

- Brak cache'u Open Library — strona szczegółów wykonuje kilkanaście żądań HTTP na odświeżenie, co wydłuża ładowanie.
- Brak CSRF — formularze nie używają tokenów CSRF (brak Flask-WTF); w środowisku produkcyjnym warto dołożyć zabezpieczenie.
- run.py seed — obecna wersja zapisuje hasła kont testowych jako plain text, dlatego zalecamy jednorazową inicjalizację bazy komendą z sekcji 2.2 (używa generate_password_hash).
- Flask-Login bez unikalności UserBookStatus — teoretycznie użytkownik może mieć wiele wpisów dla tej samej książki; w praktyce widok /update_status szuka istniejącego rekordu i go aktualizuje.

9. Źródła

9.1 Dokumentacja użytych bibliotek

- Flask — oficjalna dokumentacja frameworka
<https://flask.palletsprojects.com/>
- Flask-SQLAlchemy — integracja Flaska z SQLAlchemy
<https://flask-sqlalchemy.palletsprojects.com/>
- Flask-Login — zarządzanie sesjami użytkowników
<https://flask-login.readthedocs.io/>
- Werkzeug — w projekcie używany głównie moduł werkzeug.security (hashowanie haseł)
<https://werkzeug.palletsprojects.com/>
- Jinja2 — silnik szablonów używany przez Flask
<https://jinja.palletsprojects.com/>
- SQLAlchemy — ORM, na którym bazuje Flask-SQLAlchemy
<https://docs.sqlalchemy.org/>
- Requests — biblioteka HTTP użyta do komunikacji z Open Library
<https://requests.readthedocs.io/>
- pytest — framework testowy
<https://docs.pytest.org/>

9.2 API zewnętrzne

- Open Library Developers — opis endpointów Search, Subjects, Works, Authors, Covers
<https://openlibrary.org/developers/api>

9.3 Wzorce projektowe Flaska

- Application Factory Pattern — uzasadnienie funkcji `create_app`
<https://flask.palletsprojects.com/en/stable/patterns/appfactories/>
- Modular Applications with Blueprints — podział aplikacji na blueprinty `auth` i `books`
<https://flask.palletsprojects.com/en/stable/blueprints/>
- Larger Applications — zalecana struktura projektu
<https://flask.palletsprojects.com/en/stable/patterns/packages/>
- Testing Flask Applications — wzorce `fixture`’ów i `test_client`
<https://flask.palletsprojects.com/en/stable/testing/>

9.4 Kursy i opracowania

- Miguel Grinberg — The Flask Mega-Tutorial (rozdziały o fabryce aplikacji, Flask-Login i blueprintach)
<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-i-hello-world>
- Real Python — Discover Flask: Full Stack Web Development with Flask
<https://realpython.com/tutorials/flask/>

Małgorzata Pytlak & Julia Radosz · 2025/26